

Apuntes personales: Tópicos de ciencia de la computación

Cristóbal Alcaíno

May 14, 2026

Definition 0.1: HTML

Definimos HTML como un lenguaje de estructura semántica de documento.

Definition 0.2: DOM

Document Object Model $\implies$ Desarrollo web

Se define como el grafo $G = (V, E)$ de elementos visualizables en HTML

Caso práctico: Sea (Y) y S yo y el servidor respectivamente. Usando GET-messages envío la solicitud de que quiero mis mensajes. Luego el servidor debiese en su motor interno gestionar la solicitud y preparar el resultado $(S : JSON)$.

El DOM lo construye (Y) después de recibir $JSON(S)$.

Luego $Y : JSON \implies DOM \implies visualizar$

Quien debe paginar: Definimos el paginar como el proceso usual de cortar el JSON. Debe hacerlo (S) . Podría hacerlo (Y) pero no sería eficiente.

Definition 0.3: STATEFUL

Decimos que un servidor es STATEFUL si la tupla $(t_1, t_2, \dots, t_n)$ no influye sobre la acción A a ejecutar en t_n . Los tokens por ejemplo, no dependen de t_{n-1} . Buscar en la base de datos del server por diccionario/hash tampoco depende de t_{n-k} .

Definition 0.4: REST

Estilo arquitectónico donde manipulo recursos usando protocolos HTTP.

Satisface GET, POST, PUT, DELETE.

Es *Stateless*.

Definition 0.5: Entidad

Es un objeto del dominio. Tiene identidad propia y atributos.

Definition 0.6: PUT

Es un método de REST. Lo que hace es **reemplazar** un recurso existente.

Definition 0.7: CURSOR

Sea D un conjunto ordenado de datos. El cursor es un puntero a un dato específico desde donde continuamos la lectura.

- Es un identificador de la posición en un conjunto de datos que permite continuar la lectura desde un punto (punto específico, no índice).

Sea D tal que se modifica en LIFO-STACK. D es ordenado. Definimos el cursor como $\{e_{j\leq}\}$. Tal que si paso de D a $D \cup e$

$$\implies \{e_{j\leq}\}_{e \cup D} = \{e_{\leq}\}_D$$

Definition 0.8: OFFSET

Lo definimos de igual forma que Cursor solo que es sobre un índice. Es decir, si rellenamos por LIFO y ejecutamos la acción va a devolver un set de datos distinto.

Lemma 0.1: Pooling

Supongamos que queremos mantener una conexión instantánea. Un usuario necesita ver sus mensajes en vivo pese a que aún no llega ninguno. Puede ejecutar el GET. El servidor lo recibe. Lo deja en WAIT. Cuando llegue un mensaje el servidor lo envía. (La conexión se puede dejar en standby)

Definition 0.9: WebSocket

Protocolo de comunicación TCP persistente (en vivo por harito rato) que es bidireccional

Definition 0.10: HEAP

H es un árbol binario : H es min - heap $\iff$

$$\forall x \in H : x.sons \geq x$$

Por definición observar el mínimo es $\mathcal{O}(1)$. Extraer (es decir sacar y reordenar) es $\mathcal{O}(\log(n))$ Insertar a su vez es $\mathcal{O}(\log(n))$.

Lemma 0.2: Insertion in HEAP

Para insertar en (min-HEAP). Se inserta en la última posición disponible. Se usa el sig. algoritmo recursivo.

$$while : if not check(x.father, x) \Rightarrow swap(x.father, x)$$

Es fácil notar que la complejidad de esto es $\log(\backslash)$

0.1 Una aplicación de interés

Supongamos que tenemos un servicio de mensajes tal que cada mensaje es $M = (id, timestamp)$. Un mensaje j es más reciente que uno i si:

$$\iff t_i \leq t_j$$

. Nos gustaría tener una estructura para siempre tener los k mensajes más recientes. Uno podría pensar en un STACK pero los mensajes no llegan en perfecto orden debido a la concurrencia. Lo que nos interesa es poder manejar eficientemente la inserción. Por lo tanto hacemos un MIN-HEAP(k). Es decir en la raíz va a estar el más viejo. Luego al insertar el t_j basta comparar con la raíz (si es que el HEAP está lleno). Poppear la raíz, hacer heapify e insertar el t_j . Recordamos que las inserciones son $\mathcal{O}(\log(k))$. Si reordenásemos a

cada rato el stack tendríamos $\mathcal{O}(k \log(k))$
 Ojo que construir un HEAP por heapify-downs es $\mathcal{O}(n)$

0.2 Un problema de interés - Requests per minute

Supongamos que tenemos mensajes (id, timestamp) que llegan de forma arbitraria. Queremos que el server maneje solo 100 requests por minuto. Como lo hacemos. La solución es una ventana continua. Es decir, en cada segundo el servidor contará hasta el minuto pasado y solo mantendrá los que caen en esa ventana. El resto chao. Esa ventana va a ser de tamaño 100.

Para escalar basta tomar REDIS con operaciones atómicas. Es decir colapsar el conteo a una operación atómica.

Definition 0.11: LRU CACHE

Definimos el LRU como Least Recently Used. Es decir un stack de K elementos tal que eliminamos la wea en mayor desuso. Tal que para cada átomo a hay 2 métodos. GET(key) y PUT(key, value)

¿Como eliminamos lo que está en mayor desuso? Con una lista doble ligada.

Definition 0.12: Lista doble ligada

Es una colección de elementos tal que cada elemento tiene $x.before$ y $x.after$

Mover elementos entre sí en la lista doble ligada es $O(1)$. Depende del elemento solamente.

0.3 Algunas definiciones para modelar problemas

Definition 0.13: Evento

$e = (event_id, payload, timestamp, \dots)$
 $event_id(e_1) = event_id(e_2) \Rightarrow e_1 \sim e_2$

Definition 0.14: Estado

$state \in S | f : S \times \mathcal{E} \rightarrow S$

Definition 0.15: Invariantes

Definimos una invariante como una propiedad que debe mantenerse. La definimos a través de ejemplos:

$$\text{rate limiting} \Rightarrow \#events(u, t - \Delta, t) \leq K$$

$$\text{deduplicacion} \Rightarrow \forall e : \sum_{e' \in [e]} f(e') = 1$$

$$\text{view counting} \Rightarrow \sum_{i \in I} c_i$$

Definition 0.16: Sharding

Se define como una función de partición:

$$S : \mathcal{K} \rightarrow \{1, \dots, N\}$$

La propiedad clave es que idealmente los datos relacionados caigan en el mismo shard.

Definition 0.17: Localidad

Una operación f se define como local si: $f(state)$ depende de un solo shard.

Definition 0.18: Operación atómica

$f : \mathcal{S} \rightarrow \mathcal{S}$ tal que f no es interrumpible.

Definition 0.19: Idempotencia

$f : f(f(f(s))) = f(s)$

0.4 Datos geoespaciales

Sea $P \subset \mathbb{R}^2$ tal que queremos

$$find(p \in P : d(p, q) < r)$$

lo que debiese hacer es guardar las cosas en una grilla. Centrarme en q . Extraer el cuadrado y empezar a filtrar. Esto baja la complejidad de

$$\mathcal{O}(n) \rightarrow \mathcal{O}(\log(n) + k)$$

Proposition 0.1: Handshake lemma (SIN DEMOSTRAR AUN)

-

$$G = (V, E) \Rightarrow \sum_{n \in nodes} |n.edges| = 2M \quad \text{for some } M \in \mathbb{N}$$